LangGraph.py

Asistente de soporte · Triage → Search_Knowledge → Evaluator → Human_Support

Código

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

MAX_INTENTOS = 3

# Base de conocimiento simulada: por categoría, los artículos en el orden en que se prueban
BASE_CONOCIMIENTO = {
    "cuenta": [
        "Verificá que el correo con el que ingresás esté bien escrito.",
        "Para recuperar tu contraseña, usá 'Olvidé mi contraseña' en la pantalla de login y
seguí el enlace que te llega por mail.",
    ],
    "facturacion": [
        "Las facturas se emiten el primer día hábil de cada mes.",
        "Podés descargar tus facturas desde Mi cuenta > Pagos.",
    ],
    "general": [
        "Nuestro horario de atención es de lunes a viernes de 9 a 18.",
        "Podés ver las preguntas frecuentes en la sección Ayuda.",
        "Revisá que tu aplicación esté actualizada a la última versión.",
    ],
}

# 1. Estado: la consulta del usuario, su categoría, la respuesta encontrada y el control de
intentos
class SupportState(TypedDict):
    messages: str
    categoria: str
    respuesta: str
    resuelto: bool
    attempts: int
    is_escalated: bool

# 2. Triage: clasifica la consulta para saber en qué parte de la base de conocimiento buscar
def triage(state: SupportState) -> dict:
    texto = state["messages"].lower()
    if any(p in texto for p in ("contraseña", "login", "cuenta", "usuario")):
        categoria = "cuenta"
    elif any(p in texto for p in ("factura", "pago", "cobro")):
        categoria = "facturacion"
    else:
        categoria = "general"
    print(f"[Triage] categoría -> {categoria}")
    return {"categoria": categoria}

# 3. Search_Knowledge: en cada intento prueba el siguiente artículo de la categoría
def search_knowledge(state: SupportState) -> dict:
    counter_attempts = state["attempts"] + 1
    articulos = BASE_CONOCIMIENTO[state["categoria"]]
    respuesta = articulos[counter_attempts - 1] if counter_attempts <= len(articulos) else ""
    print(f"[Search_Knowledge] intento {counter_attempts}/{MAX_INTENTOS} -> {respuesta or
'(sin resultados)'}")
    return {"respuesta": respuesta, "attempts": counter_attempts}
```

```python
# 4. Evaluator: decide si la respuesta resuelve la consulta (la ruta la elige la arista condicional)
def evaluator(state: SupportState) -> dict:
    palabras_clave = [p for p in state["messages"].lower().split() if len(p) > 5]
    resuelto = any(p in state["respuesta"].lower() for p in palabras_clave)
    print(f"[Evaluator] resuelto -> {resuelto}")
    return {"resuelto": resuelto}

# 5. Human_Support: solo se llega acá después de agotar los intentos permitidos
def human_support(state: SupportState) -> dict:
    print(f"[Human_Support] escalado tras {state['attempts']} intentos")
    return {
        "is_escalated": True,
        "respuesta": "Tu consulta fue derivada a un agente humano, que te va a contactar a la brevedad.",
    }

# 6. Arista condicional del evaluador: resuelto, reintentar búsqueda o escalar a un humano
def route_evaluator(state: SupportState) -> str:
    if state["resuelto"]:
        return "resuelto"
    if state["attempts"] >= MAX_INTENTOS:
        return "escalar"
    return "reintentar"

# 7. Construcción del grafo
g = StateGraph(SupportState)
g.add_node("Triage", triage)
g.add_node("Search_Knowledge", search_knowledge)
g.add_node("Evaluator", evaluator)
g.add_node("Human_Support", human_support)

g.add_edge(START, "Triage")
g.add_edge("Triage", "Search_Knowledge")
g.add_edge("Search_Knowledge", "Evaluator")
g.add_conditional_edges(
    "Evaluator",
    route_evaluator,
    {"resuelto": END, "reintentar": "Search_Knowledge", "escalar": "Human_Support"},
)
g.add_edge("Human_Support", END)

app = g.compile()

# 8. Ejecución: un caso que se resuelve y uno que termina escalado
if __name__ == "__main__":
    for consulta in ("Perdi mi contraseña y no me deja ingresar", "Quiero cambiar el plan de mi empresa"):
        print(f"\n=== Consulta: {consulta}")
        final = app.invoke({
            "messages": consulta,
            "categoria": "",
            "respuesta": "",
            "resuelto": False,
            "attempts": 0,
            "is_escalated": False,
        })
        print(f"Final tras {final['attempts']} intentos | escalado: {final['is_escalated']}")
        print(final["respuesta"])
```

Diagrama del grafo

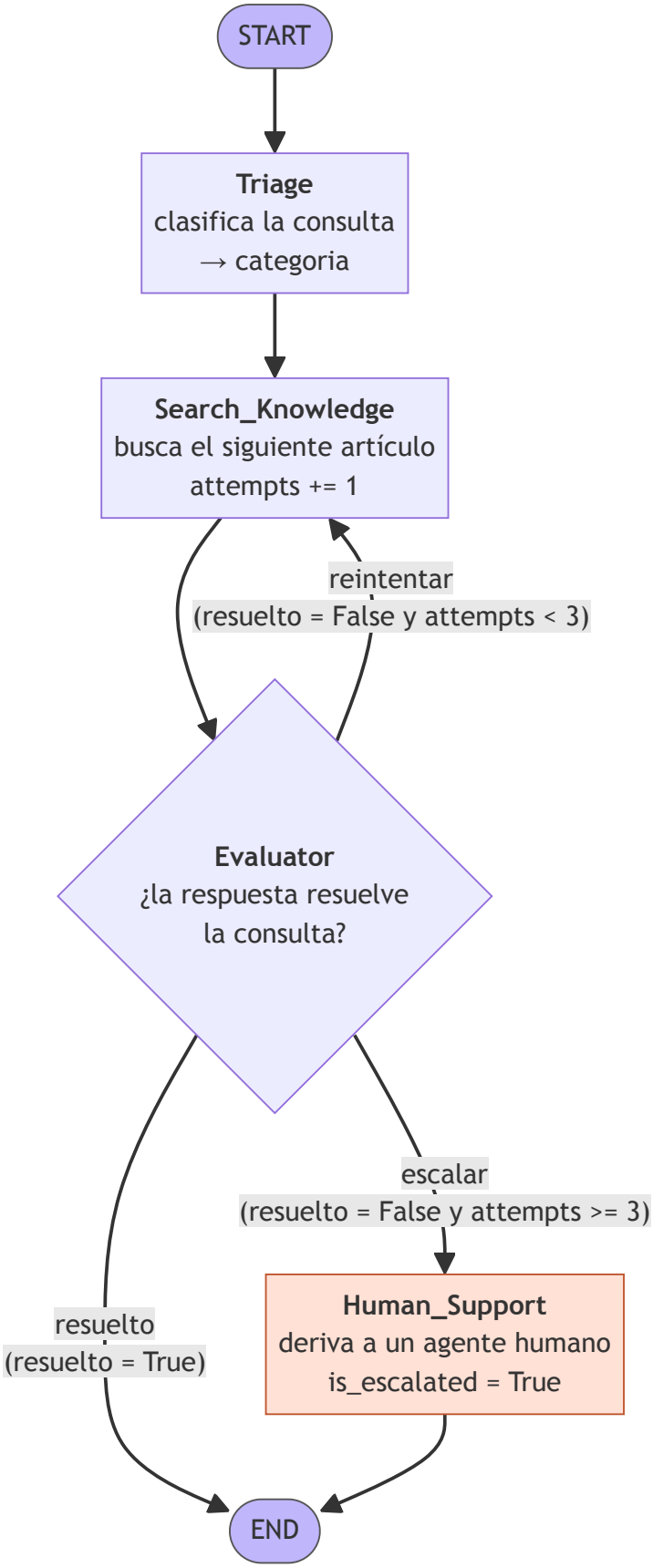
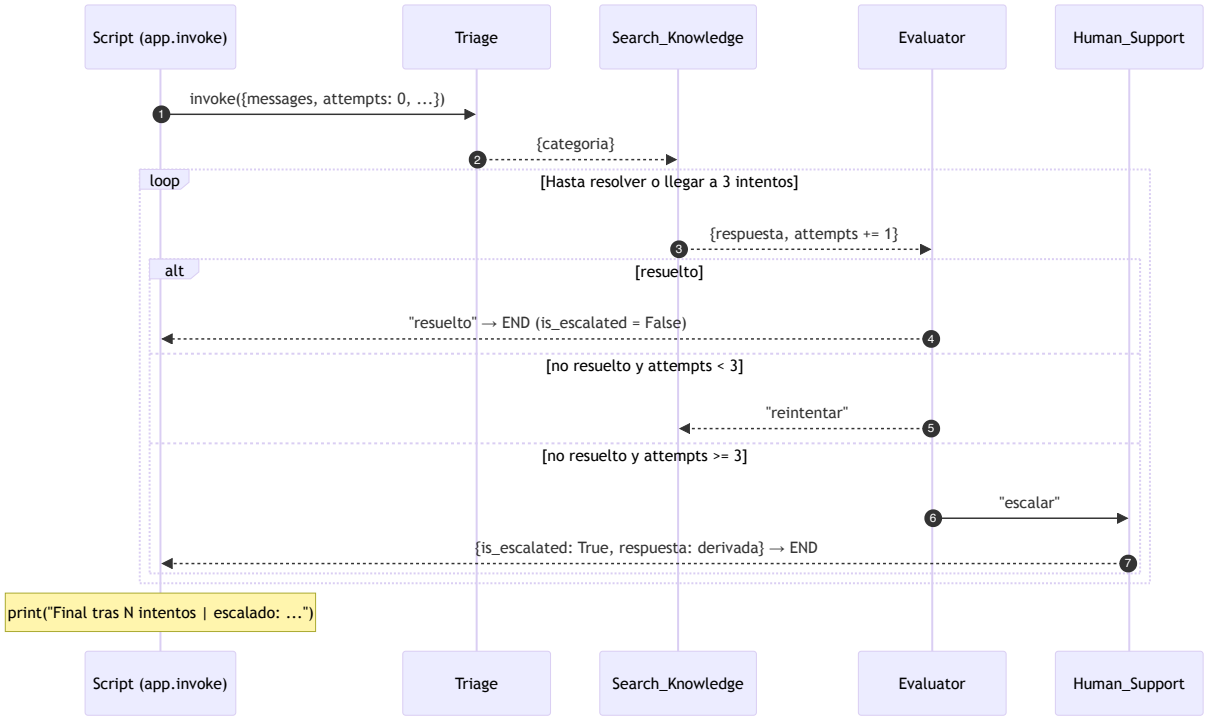


Diagrama de secuencia



Salida de ejecución

```
=== Consulta: Perdi mi contraseña y no me deja ingresar
[Triage] categoría -> cuenta
[Search_Knowledge] intento 1/3 -> Verificá que el correo con el que ingresás esté bien
escrito.
[Evaluator] resuelto -> False
[Search_Knowledge] intento 2/3 -> Para recuperar tu contraseña, usá 'Olvidé mi contraseña' en
la pantalla de login y seguí el enlace que te llega por mail.
[Evaluator] resuelto -> True
Final tras 2 intentos | escalado: False
Para recuperar tu contraseña, usá 'Olvidé mi contraseña' en la pantalla de login y seguí el
enlace que te llega por mail.

=== Consulta: Quiero cambiar el plan de mi empresa
[Triage] categoría -> general
[Search_Knowledge] intento 1/3 -> Nuestro horario de atención es de lunes a viernes de 9 a
18.
[Evaluator] resuelto -> False
[Search_Knowledge] intento 2/3 -> Podés ver las preguntas frecuentes en la sección Ayuda.
[Evaluator] resuelto -> False
[Search_Knowledge] intento 3/3 -> Revisá que tu aplicación esté actualizada a la última
versión.
[Evaluator] resuelto -> False
[Human_Support] escalado tras 3 intentos
Final tras 3 intentos | escalado: True
Tu consulta fue derivada a un agente humano, que te va a contactar a la brevedad.
```